

ppmx with tuning

Simulation Setup

```
# Controls
Nobs <- 200
Minformative <- 2
Muninformative <- 25

# simulate data

# Seed and empty dataframe
df <- tibble("row" = 1:Nobs)

# Create Minformative new variables that are random observations with apply function

for (i in 1:Minformative) {
  df <- df %>%
    mutate(
      !!paste0("Inf", i) := rnorm(Nobs),
    )
}

for (i in 1:Muninformative) {
  df <- df %>%
    mutate(
      !!paste0("Uninf", i) := rnorm(Nobs)
    )
}

df <- df %>%
  #select(-row) %>%
```

```

# sum across just Inf columns
#mutate(pRisk = rowSums(select(., starts_with("Inf")))) %>%
# Rescale pRisk to be between 0 and 1
mutate(
  #pRisk = (pRisk - min(pRisk)) / (max(pRisk) - min(pRisk)),
  pRisk = 0.5
  #    pRisk = pRisk^2
) %>%
# make sure groups are roughly balanced
#mutate(pRisk = ifelse(pRisk < median(pRisk), 0.25, 0.75)) %>%
# Create binary variable Risk group based on pRisk using rbinom
mutate(riskGroups = rbinom(Nobs, 1, pRisk)) %>%
# Create outcome variable Y that depends on all columns including riskGroups
mutate(
  # add an interaction term between Inf variables and Risk group that affects Y
  Y = (riskGroups + 1)^3 * rowSums(select(., starts_with("Inf"))) + rowSums(select(., starts_with("Inf")))
) %>%
# standardize y
mutate(
  # simulate a variable that is correlated to both Y and pRisk
  Uninf0 = Y + riskGroups + rnorm(Nobs),
  Uninf0 = (Uninf0 - mean(Uninf0)) / sd(Uninf0),
  Y = Uninf0 + Y,
  Y = (Y - mean(Y)) / sd(Y),
  Y = Y + riskGroups,
)

# create training and testing sets keeping riskGroups balanced
train <- df %>%
  group_by(riskGroups) %>%
  sample_frac(0.8) %>%
  ungroup()

test <- df %>%
  anti_join(train)

```

Joining with `by = join\_by(row, Inf1, Inf2, Uninf1, Uninf2, Uninf3, Uninf4, Uninf5, Uninf6, Uninf7, Uninf8, Uninf9, Uninf10, Uninf11, Uninf12, Uninf13, Uninf14, Uninf15, Uninf16, Uninf17, Uninf18, Uninf19, Uninf20, Uninf21, Uninf22, Uninf23, Uninf24, Uninf25, pRisk, riskGroups, Y, Uninf0)`

```

# m0,
# s20 - variance of x
# v2 - small favors partitions with similar X
# skip - k0, nu0, a0
# alpha - weight associated with cluster-variance and Gower

# Create a grid of simParms to test
simParms <- expand.grid(
  m0 = 0.5,
  s20 = c(15, 20, 25, 30),
  v2 = c(0.1, 1.0),
  k0 = 1, nu0 = 1, a0 = 1,
  alpha = 0.1,
  M= c(1/5, 1)
)

```

Model tuning and fitting

Model 1

```

# test gaussian_ppmx over this set of simParms using f1 stat
paramSelect <- simParms %>%
  rowwise() %>%
  mutate(
    simParms = list(c(m0, s20, v2, k0, nu0, a0, alpha)),
    f1 = ppmxSummary(gaussian_ppmx(
      y = train$Y,
      X = train[c(paste0("Inf", 1:Minformative), paste0("Uninf", 0:Muninformative))],
      draws = 1000, burn = 50, thin = 5, M = M, meanModel = 1, similarity_function = 1, verbose = FALSE
    )) %>%
    # turn off rowwise
    ungroup() %>%
    slice(which.max(f1)) %>%
    pull(simParms) %>%
    as.data.frame() %>%
    pull(1)

# Fit with best tuning
mod1 <- ppmSuite::gaussian_ppmx(y = test$Y, X = test[c(paste0("Inf", 1:Minformative), paste0("Uninf", 0:Muninformative))])

```

Model 2

```
# tune model with only informative variables
paramSelect3 <- simParms %>%
  rowwise() %>%
  mutate(
    simParms = list(c(m0, s20, v2, k0, nu0, a0, alpha)),
    f1 = ppmxSummary(gaussian_ppmx(y = train$Y, X = train[paste0("Inf", 1:Mininformative) ], d
  ) %>%
  # turn off rowwise
  ungroup() %>%
  slice(which.max(f1)) %>%
  pull(simParms) %>%
  as.data.frame() %>%
  pull(1)
mod3 <- ppmSuite::gaussian_ppmx(y = test$Y, X = test[paste0("Inf", 1:Mininformative)]), draws =
```

Model 3

```
# make a string summing all 1:Muninformative variables
RHS <- paste0("Uninf", 1:Mininformative, collapse = " + ")
form <- as.formula(paste("Y ~", RHS))

# Train model after projecting away Uninformative variables
train2 <- train %>%
  # residualize out the Uninf varaibles
  mutate(
    Y = lm(form, data = .)$residuals
  )
test2 <- test %>%
  # residualize out the Uninf varaibles
  mutate(
    Y = lm(form, data = .)$residuals
  )

paramSelect2 <- simParms %>%
  rowwise() %>%
  mutate(
    simParms = list(c(m0, s20, v2, k0, nu0, a0, alpha, M)),
    f1 = ppmxSummary(gaussian_ppmx(
```

```

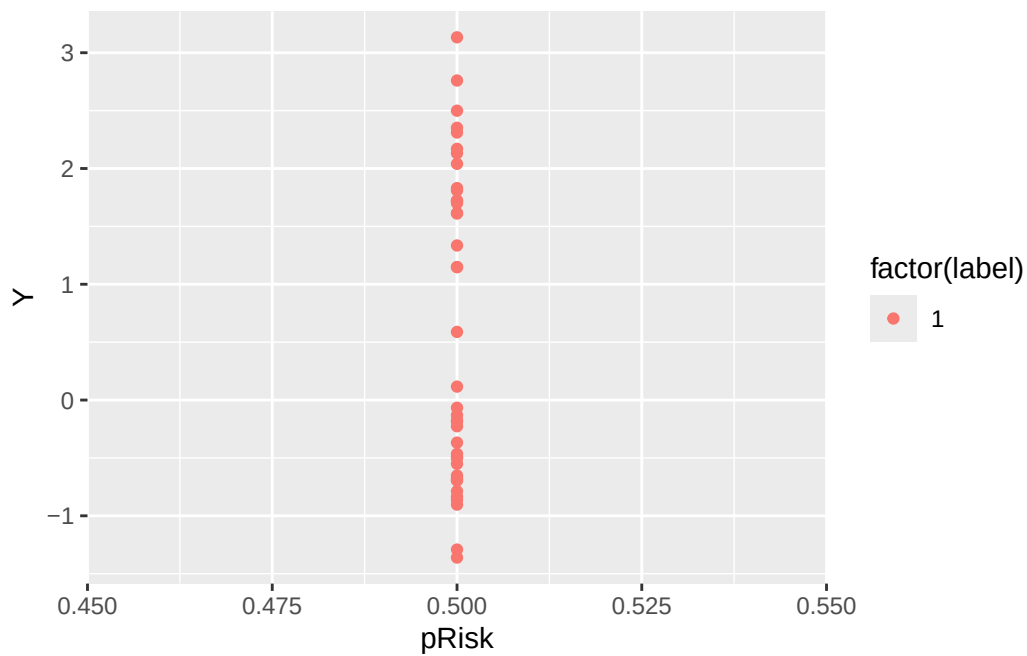
      y = train2$Y,
      X = train2[c(paste0("Inf", 1:Mininformative), "Uninf0")], draws = 1000, burn = 50, thin = 10,
    ), train2, plot=FALSE)
  ) %>%
  # turn off rowwise
  ungroup() %>%
  slice(which.max(f1)) %>%
  pull(simParms) %>%
  as.data.frame() %>%
  pull(1)

mod2 <- ppmSuite::gaussian_ppmx(y = test2$Y, X = test2[paste0("Inf", 1:Mininformative)], draws = 1000, burn = 50, thin = 10, plot=FALSE)

```

Model Evaluation

```
ppmxSummary(mod1, test, plot = T, verbose= T)
```



```

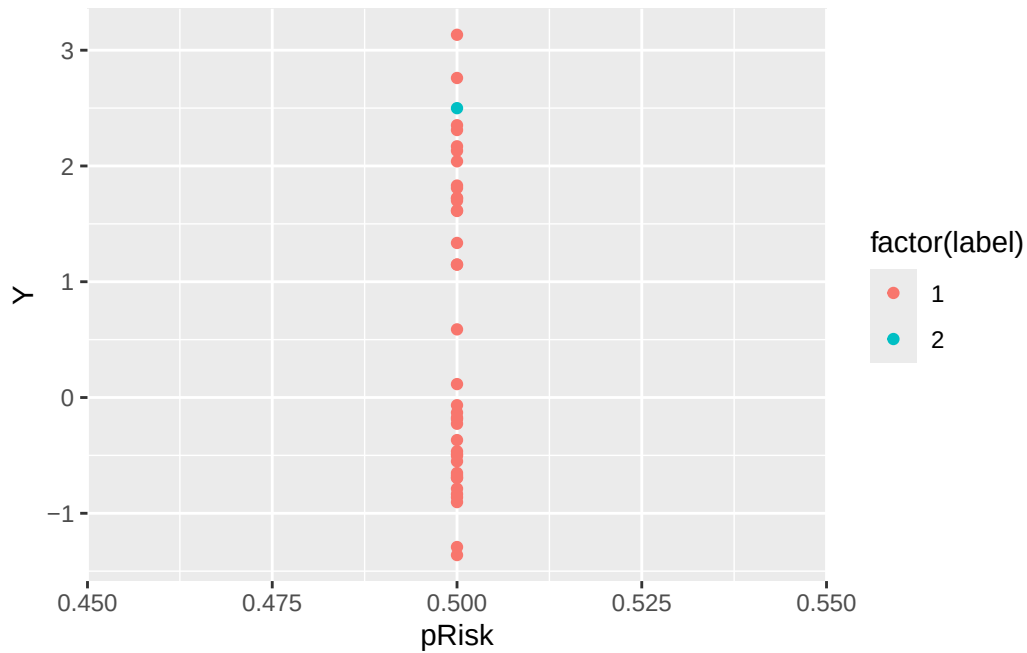
[1] "f1:0.666666666666667"
[1] "Acc:0.5"

```

```
[1] "Recall:1"
[1] "R.Index:0.487179487179487"
[1] "Precision:0.5"
[1] "Jaccard:0.0126582278481013"
```

```
[1] 0.6666667
```

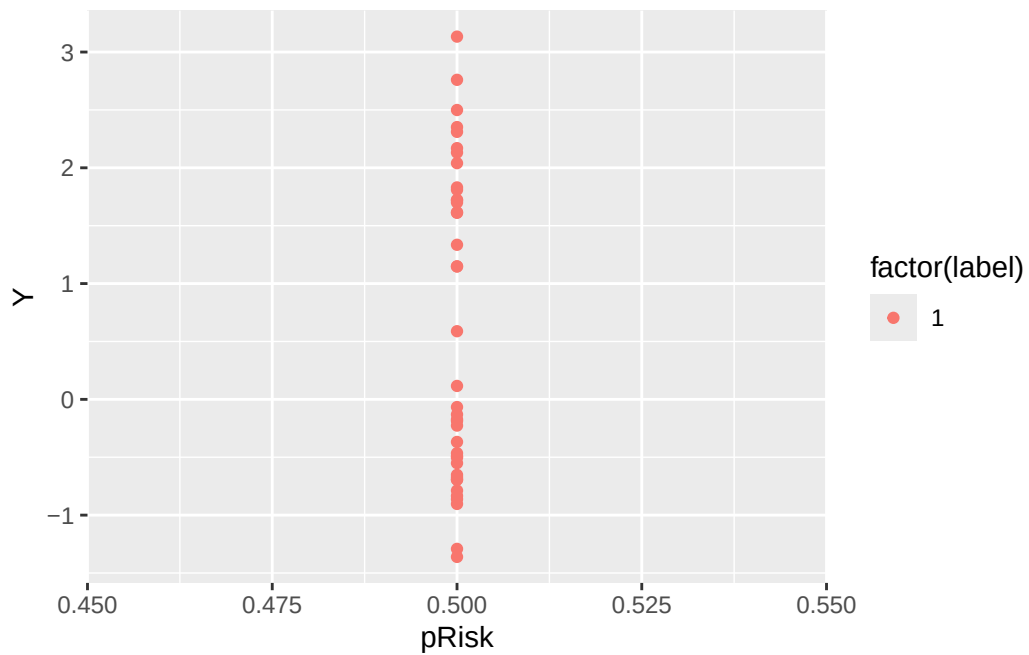
```
pplxSummary(mod2, test, plot = T, verbose= T)
```



```
[1] "f1:0.5"
[1] "Acc:0.475"
[1] "Recall:1.05"
[1] "R.Index:0.488461538461539"
[1] "Precision:0.487179487179487"
[1] "Jaccard:0.0126582278481013"
```

```
[1] 0.5
```

```
pplxSummary(mod3, test, plot = T, verbose= T)
```



```
[1] "f1:0.666666666666667"
[1] "Acc:0.5"
[1] "Recall:1"
[1] "R.Index:0.487179487179487"
[1] "Precision:0.5"
[1] "Jaccard:0.0126582278481013"
```

```
[1] 0.6666667
```

```
df %>%
  ggplot(aes(x = pRisk, y = Y, col = factor(riskGroups))) +
  geom_point()
```

